

1ACC0057 - MACHINE LEARNING

SIMULACRO

PROFESOR	Jairo Pinedo Taquia / Robert Ernesto Zubieta Cardenas
SECCIÓN	Todas
FECHA DE EVALUACIÓN	Semana 8
CICLO ACADÉMICO	xxxxxxx
DURACIÓN	110 minutos

Detección de Fraude en Transacciones Financieras

1. Descripción

El fraude financiero es una de las amenazas más costosas para el sector bancario y de pagos digitales a nivel global. En este examen, el alumno diseñará y construirá un sistema de detección de transacciones fraudulentas aplicando técnicas de Machine Learning supervisado. Se trabajará con el dataset Credit Card Fraud Detection, disponible públicamente en Kaggle (<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>). El objetivo es entrenar, evaluar y comparar tres modelos de clasificación binaria, prestando especial atención al severo desbalance de clases presente en los datos, y justificar cada decisión técnica mediante análisis escrito en celdas Markdown del notebook.

2. Conjunto de Datos

El archivo creditcard.csv contiene 284 807 transacciones realizadas con tarjeta de crédito en Europa durante septiembre de 2013. De ellas, 492 son fraudulentas ($\approx 0.17\%$). Las columnas son:

- **Time:** Número de segundos transcurridos desde la primera transacción del dataset. No tiene utilidad predictiva directa, pero puede revelar patrones temporales.
- **V1 – V28:** Componentes principales (PCA) de las variables originales, anonimizadas por confidencialidad. Son las features principales del modelo.
- **Amount:** Monto de la transacción en euros. Requiere escalado explícito.
- **Class:** Variable objetivo: 0 = transacción legítima, 1 = transacción fraudulenta.

Nota: El dataset puede descargarse con la API de Kaggle o desde el aula virtual. No se requiere ningún procesamiento de formato; se carga directamente con pandas.

3. Tareas

3.1 Análisis Exploratorio de Datos (AED)

Responder visualmente y con interpretación escrita a cada punto:

- Calcular estadísticas descriptivas (media, desviación estándar, mínimo, máximo y percentiles 25/50/75) de todas las variables numéricas.
- Visualizar la distribución de **Amount** y **Time** diferenciando transacciones legítimas y fraudulentas (histogramas y/o boxplots). Interpretar las diferencias observadas.
- Cuantificar el desbalance de clases: calcular el ratio de fraude (%), graficarlo con un diagrama de barras y argumentar cuál es su impacto en el entrenamiento si se ignora.
- Verificar la existencia de valores nulos o faltantes. Reportar el resultado y justificar la acción tomada.
- Generar un mapa de calor de correlación de las variables **V1–V28** y **Amount** con la variable **Class**. Identificar las 5 features con mayor correlación (positiva o negativa) con la clase objetivo.

3.2 Preprocesamiento de Datos

- Escalar las variables **Amount** y **Time** utilizando **StandardScaler**. Justificar en una celda Markdown por qué las variables V1–V28 no requieren escalado adicional.
- Dividir el dataset en entrenamiento (70 %), validación (15 %) y prueba (15 %), aplicando estratificación por clase (**stratify=y**). Mostrar el conteo de clases en cada split.
- Aplicar al menos una de las siguientes estrategias para manejar el desbalance de clases, justificando la elección:
 - SMOTE (Synthetic Minority Over-sampling Technique) — sobremuestreo sintético sobre el conjunto de entrenamiento.
 - RandomUnderSampler — submuestreo aleatorio de la clase mayoritaria.
 - Parámetro `class_weight='balanced'` directamente en el clasificador.
- Importante: la técnica de balanceo se aplica única y exclusivamente al conjunto de entrenamiento. Validación y prueba deben mantenerse con la distribución real.

3.3 Modelado

Implementar y entrenar tres modelos de ML, uno por cada paradigma indicado en la tabla. Cada modelo debe entrenarse sobre el conjunto de entrenamiento (con la estrategia de balanceo elegida) y generar predicciones sobre el conjunto de prueba.

Paradigma	Descripción del enfoque	Algoritmo sugerido
Aprendizaje basado en instancias	Clasifica nuevas muestras por similitud con los ejemplos de entrenamiento almacenados, sin construir un modelo explícito. Es sensible al escalado de features.	K-Nearest Neighbors (KNN)
Divide y vencerás (ensamble de árboles)	Combina múltiples árboles de decisión entrenados con sub-muestras aleatorias de datos y features, reduciendo varianza mediante votación.	Random Forest
Aprendizaje conexionista / bioinspirado	Red neuronal de tipo Perceptrón Multicapa (MLP) que aprende representaciones jerárquicas mediante retropropagación del error, inspirada en la arquitectura neuronal biológica.	MLP (MLPClassifier)

Nota: El alumno puede sustituir el algoritmo sugerido por otro del mismo paradigma, siempre que lo justifique explícitamente en el notebook.

Para cada modelo se requiere:

- Justificar en una celda Markdown por qué el algoritmo elegido pertenece al paradigma indicado.
- Documentar los hiperparámetros utilizados (valores por defecto o ajustados) y la razón de su elección.
- Generar predicciones de clase (**predict**) y de probabilidad (**predict_proba**) sobre el conjunto de prueba.
- Presentar un **cuadro comparativo final** con las métricas de los tres modelos (ver sección 3.4).

3.4 Evaluación de Modelos

Dado el severo desbalance de clases, la exactitud (accuracy) sola no es suficiente para evaluar el rendimiento. Calcular y presentar para cada modelo las siguientes métricas sobre el conjunto de prueba:

- Accuracy — *acompañar con una breve reflexión sobre su limitación en contextos muy desbalanceados.*
- Precision, Recall y F1-score por clase y macro-promedio (usar **classification_report** de scikit-learn).

- AUC-ROC y curva ROC de los tres modelos graficada en un mismo plot, con leyenda e interpretación.
- AUC-PR (Precision-Recall Curve) — graficar y argumentar por qué esta métrica es más informativa que la ROC cuando el desbalance es severo.
- Matriz de Confusión de cada modelo, con interpretación explícita de los Falsos Positivos (cliente legítimo bloqueado) y Falsos Negativos (fraude no detectado) en términos de impacto de negocio.

El cuadro comparativo final debe consolidar todas las métricas anteriores en un único DataFrame mostrado con `display()` o `print()` en el notebook, con los tres modelos como columnas o filas.

3.5 Análisis y Conclusiones

- Comparar los tres modelos con base en AUC-ROC, AUC-PR y F1-score de la clase minoritaria (clase 1). ¿Cuál modelo es más adecuado para un entorno de producción? Justificar.
- Analizar el trade-off Precisión vs. Recall para el modelo ganador. ¿Qué umbral de decisión recomendarías para minimizar pérdidas por fraude no detectado? Justificar con la curva PR.
- Identificar las **features más importantes** según Random Forest (atributo `feature_importances_`). ¿Coinciden con las variables de mayor correlación halladas en el AED? Discutir.
- Señalar al menos **dos limitaciones** del sistema construido y proponer estrategias concretas de mejora (por ejemplo: modelos de detección de anomalías, reentrenamiento periódico, monitoreo de data drift, uso de datos de flujo en tiempo real).
- Reflexionar brevemente sobre las implicaciones éticas del sistema: ¿qué consecuencias tiene un alto número de Falsos Positivos sobre clientes reales? ¿Cómo balancear la detección de fraude con la experiencia del usuario?

Presentación del Examen

- Entregar el notebook en formato **CÓDIGO_DE_ALUMNO.ipynb** (ejemplo: U20210001.ipynb).
- Cada sección debe incluir su código ejecutado inmediatamente después del enunciado, acompañado de celdas Markdown con la interpretación de los resultados.
- Los gráficos deben incluir título, etiquetas de ejes y leyenda cuando corresponda.
- Se valorará la claridad del código (comentarios, nombres de variables descriptivos) y la profundidad del análisis escrito.

Lineamientos

✓ Permitido	✗ Prohibido
Apuntes personales y material del aula virtual.	Uso de herramientas de IA generativa (ChatGPT, Copilot, Gemini, etc.). La detección implica nota 0.
Librerías Python estándar: scikit-learn, imbalanced-learn, matplotlib, seaborn, pandas, numpy.	Compartir código o resultados con otros alumnos durante el examen.
Consultar documentación oficial de las librerías.	Acceder a soluciones públicas del dataset en Kaggle, GitHub o foros.

💡 Recuerda: en detección de fraude, un Falso Negativo (fraude no detectado) puede ser mucho más costoso que un Falso Positivo. Diseña tu sistema pensando en el impacto real del negocio.